

# Testrix

## Server & Infrastructure Requirements For Company Data Center / Server Team

|              |                                                             |
|--------------|-------------------------------------------------------------|
| Prepared for | Company Server / Infrastructure Team                        |
| Purpose      | Provisioning a server to host the Testrix platform          |
| Domain       | Provided by application owner (already purchased)           |
| Access model | Limited / credential-based access on company-managed server |
| Date         | 2026-09-08                                                  |

## 1. What Testrix Is (brief context for the server team)

---

Testrix is a multi-service web application delivered as a set of Docker containers behind a single reverse-proxy gateway. It includes three backend APIs, a web frontend, a MySQL database, an automation/browser-test execution engine, and a background job scheduler that runs **inside the application itself** — no separate cron infrastructure is required from the server side. Everything is deployed via Docker, so the server does not need Java, Node.js, or any application build tools installed — only Docker.

## 2. Server Specification (request this configuration)

---

| Item        | Minimum                                                                                                      | Recommended (asking for this)                   |
|-------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| OS          | Linux — Ubuntu 22.04 / 24.04 LTS (or the team's standard distro; any modern Linux with Docker support works) |                                                 |
| CPU         | 2 vCPU                                                                                                       | <b>4 vCPU — dedicated, not shared/burstable</b> |
| RAM         | 4 GB                                                                                                         | <b>8 GB</b>                                     |
| Disk        | 40 GB                                                                                                        | <b>80 GB, SSD/NVMe preferred</b>                |
| Public IPv4 | Required — the domain must resolve to this server                                                            |                                                 |

**Why dedicated CPU matters:** Testrix runs real browser automation (Selenium/Playwright) for automated testing. A live Chrome session is CPU-heavy, not just memory-heavy. Shared/burstable CPU tiers throttle hard under this kind of sustained load — a dedicated allocation avoids unpredictable slowdowns.

## 3. Software to Install (or access to install it)

---

- **Docker Engine + Docker Compose plugin** — this is the only required software. The application ships as pre-built Docker images.
- No Java, Node.js, Python, or database server needs to be installed separately — everything runs inside containers, including the MySQL database.

If the server account we're given does *not* have sudo/install rights, please install Docker + Compose for us as a one-time setup step, and add our account to the docker group so we can manage containers without further sudo access.

## 4. Access Requested

---

| Access      | Details                                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| SSH         | Key-based login (no password auth) to a Linux user account                                                                                   |
| Docker      | That user must be able to run <code>docker / docker compose</code> commands (either via the <code>docker</code> group or <code>sudo</code> ) |
| Persistence | Confirmation that our home directory / a mounted data path survives server reboots and any snapshot/redeploy cycles                          |

**Question for the server team:** what tier of access will we actually receive — full `sudo`, a `docker-group`-only account, or something more restricted (e.g. deployment via a portal, no shell)? This determines whether steps in Sections 3 and 5 are ours to do or the team's.

## 5. Network & Firewall

| Port | Direction | Purpose                                                             |
|------|-----------|---------------------------------------------------------------------|
| 22   | Inbound   | SSH — restrict to our office/VPN IP if possible                     |
| 80   | Inbound   | HTTP (redirects to HTTPS; also used for SSL certificate validation) |
| 443  | Inbound   | HTTPS — all real traffic                                            |

**Nothing else needs to be opened inbound.** The database, cache, and internal backend services communicate only inside Docker's internal network and must never be exposed to the public internet.

### Outbound access needed (easy to miss in locked-down data centers)

- **Docker Hub** (or the team's internal image registry) — to pull application images
- **SMTP (port 587/465)** — the application sends email notifications
- **HTTPS (443) to third-party AI APIs** — used by the in-app AI assistant
- **HTTPS (443) to Let's Encrypt** — only if we manage our own SSL certificate (see Section 6)

Please confirm the data center's outbound firewall policy. If outbound traffic is restricted by default, we'll need explicit allow-rules for the destinations above.

## 6. SSL / TLS

**Question for the server team:** is TLS already terminated for us at a shared load balancer / reverse proxy in front of company-hosted servers (common in corporate data centers)? If so, we don't need to manage certificates ourselves — just confirm traffic reaches our container over plain HTTP internally.

If not, we will self-manage via Let's Encrypt (free, auto-renewing), which only requires port 80 to be reachable from the internet for domain validation — no extra action needed from the server team beyond opening that port.

## 7. Persistent Storage

---

The following must survive container restarts and server reboots — please confirm the disk backing our server is not ephemeral (i.e. not wiped on redeploy/restart):

- Database data (MySQL)
- Automation test reports, screenshots, and execution logs
- Uploaded files and generated documents

## 8. Background Jobs / Scheduler — no extra infrastructure needed

Testrix has its own internal **scheduler** (recurring API test runs, automation execution queue processing, cleanup jobs) that runs as part of the application process itself — it is **not** a separate service and does **not** require OS-level cron, a message queue, or any additional software from the server team.

**One implication to flag now for future scaling:** because the scheduler runs in-process, our application containers must run as a *single instance* each. If we ever ask to scale a backend to multiple replicas for more capacity, the scheduled jobs would fire once per replica (duplicate runs) unless we've first made that change on our side. This is on us to solve in the application, not something the server team needs to plan for today — flagging it only so it's on record for the future scaling conversation in Section 9.

## 9. Scaling & Future Space Considerations

The server sized in Section 2 comfortably covers the initial staging rollout. As usage grows, the following are the realistic next asks — flagging them now so the server team can plan capacity rather than being surprised later:

| Trigger                                  | Likely next request                                                                                                                                                                                                                                                                         |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Disk filling up                          | Test reports, screenshots, logs, and database size grow continuously with usage. Request either a larger disk allocation up front or confirmation of how quickly additional storage can be added later (self-service vs. ticket-based, and typical turnaround time).                        |
| More concurrent automated test runs      | Each concurrent browser-automation run needs meaningful dedicated CPU/RAM. If usage grows past ~2–4 concurrent runs, we'll likely ask for more CPU/RAM on this server, or a <b>second server</b> dedicated just to running automated tests (keeping the main application server untouched). |
| Production environment alongside staging | Once this environment is stable, we may request a second, fully separate server for production — separate from staging so the two never compete for resources or risk each other's stability.                                                                                               |
| General traffic growth                   | Standard vertical scaling (more CPU/RAM on the same box) before any architectural change is needed — no redesign required on our side to absorb this.                                                                                                                                       |

**Ask the server team directly:** what is the process and typical turnaround time to increase CPU/RAM/disk on an already-provisioned server, or to provision an additional server, once we need to scale? Knowing this now avoids delays later.

## 10. Backup

We will run our own scheduled database backups from inside the application. Please confirm whether the data center separately provides VM-level snapshots/backups as a second line of defense, and how far back those are retained.

## 11. Summary Checklist for the Server Team

---

- ☐ Linux server — 4 vCPU (dedicated) / 8 GB RAM / 80 GB SSD, public IPv4
- ☐ Docker Engine + Docker Compose plugin installed
- ☐ SSH key-based access to a user account, with Docker permissions
- ☐ Inbound firewall: ports 22 (restricted), 80, 443 only
- ☐ Outbound access confirmed for: Docker Hub, SMTP, third-party API HTTPS calls, Let's Encrypt (if applicable)
- ☐ Confirmation: SSL terminated upstream, or port 80 open for Let's Encrypt validation
- ☐ Confirmation: disk is persistent across reboots/redeploys
- ☐ Confirmation: VM-level backup/snapshot policy, if any
- ☐ Clarified: process & turnaround time for future CPU/RAM/disk increases or a second server

Testrix — Server Requirements Document · Prepared for handoff to company server/infrastructure team